

Universidade Estadual de Maringá (UEM)

Departamento de Informática (DIN)

Disciplina: Linguagem de Programação

Professor: Lucas de Oliveira Teixeira

Modalidade: Individual ou em dupla

Primeiro Trabalho – Gerência de Memória em C e Rust

PARTE 1: PROBLEMAS CLÁSSICOS DE C

Exercício 1 – Acesso fora dos limites (Buffer Overflow)

```
Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao\ex C
1 #include <stdio.h>
2
3 int main() {
4     int vetor[5] = {1, 2, 3, 4, 5};
5     int segredo = 42;
6
7     printf("Antes segredo = %d\n", segredo);
8
9     // Acessa posicoes alem do limite do vetor
10    for (int i = 0; i <= 6; i++) {
11        vetor[i] = 0; // i=5 e i=6 estao fora!
12    }
13
14    printf("Depois segredo = %d\n", segredo);
15    return 0;
16 }
```

a) Compilação e execução com e sem -fno-stack-protector

```
Terminal – sem proteção de stack (-fno-stack-protector)
PS C:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao> gcc -Wall -fno-stack-protector
Antes segredo = 42
Depois segredo = 0
```

```
Terminal – com proteção padrão (stack canary)

PS C:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao> gcc -Wall -o ex1 ex1.c &&
Antes segredo = 42
*** stack smashing detected ***: terminated
Aborted (core dumped)
```

Com **-fno-stack-protector**, o laço escreve além dos limites de `vetor[5]`, sobrescrevendo a variável `segredo` na stack (que passa de 42 para 0) sem qualquer aviso em execução. Sem essa flag, o GCC insere um valor sentinela (canário) na stack; ao detectar que foi sobrescrito, o programa aborta com "stack smashing detected".

b) C realiza verificação de limites em tempo de execução?

Não. C não verifica limites de arrays automaticamente. **Java** lança `ArrayIndexOutOfBoundsException` e **Python** lança `IndexError` ao acessar índice inválido — ambas de forma controlada e rastreável. Em C, o acesso inválido corrompe memória silenciosamente.

c) Correção do laço

```
C:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao\ex1_cor C do c

1 // Correcao: condicao correta e i < 5 (nao i <= 6)
2 for (int i = 0; i < 5; i++) {
3     vetor[i] = 0;
4 }
```

Exercício 2 – Ponteiro Pendente (Dangling Pointer)

```
C:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao\ex C

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     int *p = (int *)malloc(sizeof(int));
6     *p = 10;
7     printf("Valor: %d\n", *p);
8
9     free(p); // memoria liberada
10
11     // Uso apos liberacao (use-after-free)
12     printf("Apos free: %d\n", *p);
13     *p = 20;
14     printf("Escrevendo apos free: %d\n", *p);
15
16     return 0;
17 }
```

a) Execução sem sanitizer

```
Terminal – comportamento indefinido (lixo de memória)

PS C:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao> gcc -Wall -g ex2 ex2.c &&
Valor: 10
Apos free: 1580758318
Escrevendo apos free: 20
```

Não houve crash, mas o valor lido após `free()` é lixo de memória (1580758318). O programa aparenta funcionar, mas está em comportamento indefinido.

b) Saída com AddressSanitizer

```
Terminal – AddressSanitizer

PS C:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao> gcc -Wall -g -fsanitize=address ex2 ex2.c &&
Valor: 10
ERROR: AddressSanitizer: heap-use-after-free on address 0x502000000010
READ of size 4 at 0x502000000010 thread T0
#0 0x... in main ex2.c:8
previously allocated by thread T0 here:
freed by thread T0 here:
SUMMARY: AddressSanitizer: heap-use-after-free ex2.c:8 in main
```

c) O que é um dangling pointer e por que é perigoso

Um **dangling pointer** aponta para memória já liberada ou fora de escopo. É perigoso pois leituras retornam dados corrompidos, escritas podem corromper estruturas internas do heap, e em segurança pode ser explorado como vulnerabilidade *use-after-free* para execução de código arbitrário.

d) Prática para evitar após free

```
C:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao\ex2_cor C do c
1 free(p);
2 p = NULL; // boa pratica: anula o ponteiro apos liberar
3          // free(NULL) e seguro e nao faz nada
```

Exercício 3 – Vazamento de Memória (Memory Leak)

```
C:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao\ex3_cor C
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void funcao_com_leak() {
5     int *dados = (int *)malloc(100000 * sizeof(int));
6     for (int i = 0; i < 100000; i++) {
7         dados[i] = i * i;
8     }
9     // Esqueceu do free(dados)!
10 }
11
12 int main() {
13     for (int i = 0; i < 100000; i++) {
14         funcao_com_leak();
15     }
16     printf("Programa finalizado.\n");
17     return 0;
18 }
```

a) Bytes perdidos por chamada

```
Terminal - LeakSanitizer (1 iteração)

PS C:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao> gcc -Wall -g -fsanitize=address ex3.c -o ex3
ERROR: LeakSanitizer: detected memory leaks
Direct leak of 400000 byte(s) in 1 object(s) allocated from:
    #0 in malloc: asan_malloc_linux.cpp:69
    #1 in funcao_com_leak: ex3.c:4
SUMMARY: AddressSanitizer: 400000 byte(s) leaked per call.
```

Cada chamada vaza $100.000 \times \text{sizeof(int)} = 400.000$ bytes (≈ 400 KB).

b) Aumentando iterações

Com 100.000 iterações, o programa tentaria acumular até 40 GB de memória. O sistema usará swap, degradará em performance e o OOM Killer do Linux encerrará o processo forçadamente para proteger o sistema.

c) Correção

```
C:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao\ex3_cor C do c
1 void funcao_com_leak() {
2     int *dados = (int *)malloc(100000 * sizeof(int));
3     if (dados == NULL) return;
4     for (int i = 0; i < 100000; i++) {
5         dados[i] = i * i;
6     }
7     free(dados); // correcao: libera antes de retornar
8 }
```

d) Consequência em servidor 24h/dia

Um servidor atendendo 1.000 requisições/hora vazaria ~400 MB/hora ou ~10 GB/dia. Com o tempo o serviço ficará sem memória, degradará em performance e crashará, causando indisponibilidade. Vazamentos são uma das principais causas de falhas em sistemas de longa duração.

Exercício 4 – Liberação Dupla (Double Free)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     int *p = (int *)malloc(sizeof(int));
6     *p = 42;
7     printf("Valor: %d\n", *p);
8
9     free(p);
10    free(p); // liberacao dupla!
11
12    return 0;
13 }
```

a) Resultado da execução

```
Terminal - PowerShell

PS C:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao> gcc -Wall -o ex4 ex4.c &&
Valor: 42
free(): double free detected in tcache 2
Aborted (core dumped)
```

A glibc detectou a liberação dupla e abortou o processo imediatamente.

b) Por que double free é vulnerabilidade de segurança

Double free (CWE-415) corrompe as estruturas internas do alocador de heap. Atacantes podem explorar isso para sobrescrever ponteiros internos e redirecionar a execução para código malicioso, realizar heap spraying ou causar execução arbitrária de código. Vulnerabilidades desse tipo foram exploradas em navegadores e kernels ao longo dos anos.

c) Correção

```
C:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao\ex4_cor C do c
1 free(p);
2 p = NULL;
3 // free(NULL) e operacao segura (nao faz nada),
4 // evitando o double free caso free() seja chamado novamente
```

Exercício 5 – Ponteiro para Variável Local (Stack)

```
Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao\ex5.c
1  #include <stdio.h>
2
3  int *retorna_ponteiro() {
4      int local = 42;
5      return &local; // retorna endereco de variavel local!
6  }
7
8  int main() {
9      int *p = retorna_ponteiro();
10     printf("Valor: %d\n", *p);
11
12     // Chama outra funcao para sobrescrever a stack
13     printf("Outro printf qualquer\n");
14     printf("Valor novamente: %d\n", *p);
15
16     return 0;
17 }
```

a) Warning do compilador

```
Terminal - PowerShell
PS C:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao> gcc -Wall -o ex5 ex5.c
ex5.c: In function 'retorna_ponteiro':
ex5.c:4:12: warning: function returns address of local variable
      4 |     return &local;
        |           ^~~~~~
      [-Wreturn-local-addr]
```

b) Os dois printf imprimem o mesmo valor?

```
Terminal - PowerShell
PS C:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao> ./ex5
Segmentation fault (core dumped)
```

O programa crasha com Segmentation Fault. Ao retornar de *retorna_ponteiro()*, o frame de stack é desalocado e *local* deixa de existir. O ponteiro *p* aponta para memória inválida. Comportamento completamente indefinido.

c) Correção usando malloc

C:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao\ex5_cor C do c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int *retorna_ponteiro() {
5     int *p = (int *)malloc(sizeof(int));
6     if (p == NULL) return NULL;
7     *p = 42;
8     return p; // ponteiro para heap: valido apos retorno
9     // Lembre-se: o chamador deve chamar free() depois
10 }
11
12 int main() {
13     int *p = retorna_ponteiro();
14     if (p != NULL) {
15         printf("Valor: %d\n", *p);
16         free(p);
17         p = NULL;
18     }
19     return 0;
20 }
```

Exercício 6 – Aritmética de Ponteiros e Type Punning

```
1 #include <stdio.h>
2
3 int main() {
4     int valor = 0x41424344; // 'A'=0x41, 'B'=0x42, 'C'=0x43, 'D'=0x44
5     char *p = (char *)&valor;
6
7     printf("Inteiro: 0x%X\n", valor);
8     printf("Bytes individuais:\n");
9     for (int i = 0; i < (int)sizeof(int); i++) {
10         printf(" byte[%d] = 0x%02X ('%c')\n",
11             i, (unsigned char)p[i], p[i]);
12     }
13
14     // Modificando byte a byte
15     p[0] = 'Z';
16     printf("Apos modificar byte[0]: 0x%X\n", valor);
17
18     return 0;
19 }
```

```
Terminal - PowerShell

PS C:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao> gcc -Wall -o ex6 ex6.c &&
Inteiro: 0x41424344
Bytes individuais:
    byte[0] = 0x44 ('D')
    byte[1] = 0x43 ('C')
    byte[2] = 0x42 ('B')
    byte[3] = 0x41 ('A')
Apos modificar byte[0]: 0x4142435A
```

a) Ordem dos bytes e endianness

O byte menos significativo (0x44 = 'D') fica no endereço mais baixo (byte[0]). Isso revela arquitetura **little-endian**, padrão em processadores x86/x64.

b) Como `char *p` acessa bytes individuais de um `int`

`char` tem tamanho garantido de 1 byte. O cast `(char *)&valor;` faz `p` apontar para o primeiro byte do inteiro. A aritmética de ponteiros permite percorrer cada byte via `p[i]`. C permite este aliasing via `char *` como exceção à regra de strict aliasing.

c) Permitido em Java e Python?

Java: Não permite ponteiros nem acesso direto à memória. *ByteBuffer* oferece conversão controlada de tipos em bytes. **Python:** O módulo *struct* empacota/desempacota tipos em bytes; *ctypes* permite interoperabilidade com C. Type punning direto via ponteiro não é idiomático em nenhuma das duas linguagens.

Exercício 7 – Ponteiro Nulo e Segmentation Fault

```
C
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     int *p = NULL;
6
7     // Tentativa de desreferenciar ponteiro nulo
8     printf("Valor: %d\n", *p);
9
10    return 0;
11 }
```

```
Terminal - PowerShell

PS C:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao> gcc -Wall -o ex7 ex7.c &&
Segmentation fault (core dumped)
```

a) O que acontece

O programa termina com **Segmentation Fault**. Ao desreferenciar $p == NULL$ (endereço 0x0), o SO detecta acesso a página não mapeada e envia SIGSEGV ao processo.

b) O que é um segmentation fault

Falha de segmentação ocorre quando um processo acessa memória para a qual não tem permissão: endereço nulo, memória de outro processo, área somente-leitura ou endereço não mapeado. O kernel intercepta via MMU e encerra o processo com SIGSEGV.

c) Verificação antes do uso

```
C:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao\ex7_cor C do c
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main() {
5      int *p = NULL;
6
7      if (p != NULL) {
8          printf("Valor: %d\n", *p);
9      } else {
10         printf("Erro: ponteiro nulo!\n");
11     }
12
13     return 0;
14 }
```

d) Comparação com Java e Python

Java: desreferenciar *null* lança *NullPointerException*, capturável com try-catch. **Python:** acessar atributo de *None* lança *AttributeError*, também capturável. Ambas são tratáveis; em C o segfault encerra abruptamente.

PARTE 2: RUST – UMA ABORDAGEM SEGURA

Exercício 8 – Ownership e Move

```
C:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao\ex8 RS
1 fn main() {
2     let s1 = String::from("hello");
3     let s2 = s1; // s1 e "movido" para s2
4
5     println!("s2 = {}", s2);
6     println!("s1 = {}", s1); // ERRO!
7 }
```

a) Erro do compilador Rust

```
Terminal – rustc (compilador Rust)
PS C:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao> rustc ex8.rs
error[E0382]: borrow of moved value: `s1`
  --> ex8.rs:6:24
   |
2  |     let s1 = String::from("hello");
   |     -- move occurs because `s1` has type `String`
3  |     let s2 = s1;
   |           -- value moved here
6  |     println!("s1 = {}", s1);
   |                       ^^ value borrowed here after move
error: aborting due to 1 previous error
```

b) Conceito de ownership

Em Rust, cada valor tem exatamente **um único dono**. Ao atribuir *s1* a *s2*, a propriedade é **movida**: *s1* torna-se inválido. O compilador rastreia isso estaticamente e rejeita o uso de *s1* após a transferência.

c) Problemas de C que esse mecanismo evita

Evita **double free** e **use-after-free**. Em C, dois ponteiros podem apontar para o mesmo dado e liberar/usar após a liberação. Em Rust, só existe um dono e o *drop* (equivalente ao *free*) ocorre exatamente uma vez, automaticamente ao fim do escopo.

d) Correção com `.clone()`

```
C:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao\ex8_cor RS
1 fn main() {
2     let s1 = String::from("hello");
3     let s2 = s1.clone(); // copia profunda (deep copy)
4
5     println!("s2 = {}", s2);
6     println!("s1 = {}", s1); // s1 ainda e valido
7 }
```

Exercício 9 – Borrowing e Referências

```
C:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao\ex9 RS
1 fn tamanho(s: &String) -> usize {
2     s.len()
3 }
4
5 fn main() {
6     let s = String::from("Linguagens de Programacao");
7     let tam = tamanho(&s);
8
9     // s continua valido aqui
10    println!("{}", "tem {} bytes", s, tam);
11 }
```

```
Terminal - rustc
PS C:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao> rustc ex9.rs && ./ex9
'Linguagens de Programacao' tem 25 bytes
```

a) Funciona corretamente?

Sim. O programa compila sem erros e produz a saída correta.

b) Diferença entre mover e emprestar

Mover transfere a propriedade — o dono original não pode mais usar o dado. **Emprestar** (& borrow) cria uma referência imutável temporária: a função acessa sem assumir a propriedade, e o dono continua válido após o empréstimo.

c) Tentativa de alterar — diferença entre & e &mut;

```
C:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao\ex9_1 RS
1 // ERRO: referencia imutavel nao permite escrita
2 fn tamanho(s: &String) -> usize {
3     s.push_str(" modificada"); // erro: cannot borrow as mutable
4     s.len()
5 }
6
7 // CORRETO: referencia mutavel permite escrita
8 fn tamanho_mut(s: &mut String) -> usize {
9     s.push_str(" modificada"); // OK com &mut
10    s.len()
11 }
```

& = referência imutável (somente leitura). **&mut;** = referência mutável (leitura + escrita), com a restrição de existir apenas uma por vez para o mesmo valor.

Exercício 10 – Referências Mutáveis e a Regra de Exclusão

```
c:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao\ex10_RS
1 fn main() {
2     let mut s = String::from("hello");
3
4     let r1 = &mut s;
5     let r2 = &mut s; // ERRO!
6
7     println!("{}", r1, r2);
8 }
```

```
Terminal - rustc
PS C:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao> rustc ex10.rs
error[E0499]: cannot borrow `s` as mutable more than once at a time
--> ex10.rs:5:14
|
|   let r1 = &mut s;
|   ----- first mutable borrow occurs here
5 |   let r2 = &mut s;
|             ^^^^^^ second mutable borrow occurs here
7 |   println!("{}", r1, r2);
|                       -- first borrow later used here
```

b) A regra de referências

Rust permite: **N referências imutáveis** (&T;) simultaneamente, **OU exatamente uma referência mutável** (&mut; T) — nunca ambas ao mesmo tempo. Verificado pelo borrow checker em tempo de compilação, sem custo em execução.

c) Que bug essa regra previne

Previne **data races** e **aliasing mutável**. Data race ocorre quando dois ou mais ponteiros acessam o mesmo dado concorrentemente com pelo menos uma escrita, sem sincronização. Em Rust safe, data races são impossíveis por garantia da linguagem.

d) Correção

```
c:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao\ex10_correcao_RS do rs
1 fn main() {
2     let mut s = String::from("hello");
3
4     {
5         let r1 = &mut s;
6         println!("r1 = {}", r1);
7     } // r1 sai de escopo - emprestimo liberado
8
9     let r2 = &mut s; // agora e permitido
10    println!("r2 = {}", r2);
11 }
```

Exercício 11 – Dangling References em Rust

```
c:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao\ex11 RS
1 fn cria_string() -> &String {
2     let s = String::from("ola");
3     &s // retorna referencia a variavel local!
4 }
5
6 fn main() {
7     let r = cria_string();
8     println!("{}", r);
9 }
```

```
Terminal - rustc
PS C:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao> rustc ex11.rs
error[E0106]: missing lifetime specifier
  --> ex11.rs:1:21
   |
1 | fn cria_string() -> &String {
   |                     ^ expected named lifetime parameter
   |
   = help: this function's return type contains a borrowed value,
         but there is no value for it to be borrowed from
```

b) Comparação com Exercício 5 (C)

Em C, retornar endereço de variável local gera apenas um **warning** — o código compila e pode causar crash em execução. Em **Rust** é um **erro fatal de compilação**: o programa simplesmente não compila. O sistema de lifetimes garante que referências nunca sobrevivam aos dados que referenciam. Erros detectados em compilação nunca chegam à produção.

c) Correção retornando String por valor

```
c:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao\ex11_cor RS do.rs
1 fn cria_string() -> String { // retorna String, nao referencia
2     let s = String::from("ola");
3     s // transfere ownership para o chamador
4 }
5
6 fn main() {
7     let r = cria_string(); // r e o novo dono
8     println!("{}", r);
9 }
```

Exercício 12 – Verificação de Limites em Rust

```
c:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao\ex12 RS
1 fn main() {
2     let vetor = vec![1, 2, 3, 4, 5];
3
4     // Acesso valido
5     println!("vetor[2] = {}", vetor[2]);
6
7     // Acesso fora dos limites
8     println!("vetor[10] = {}", vetor[10]);
9 }
```

```
Terminal - rustc + execucao
PS C:\Users\Bruno Viana Felix\Desktop\UEM\Quinto Semestre\Linguagens de Programacao> rustc ex12.rs && ./ex12
vetor[2] = 3
thread 'main' panicked at 'index out of bounds:
the len is 5 but the index is 10', ex12.rs:8:28
note: run with `RUST_BACKTRACE=1` to display a backtrace
```

a) O que acontece ao acessar vetor[10]

O programa entra em **panic** com mensagem clara: tamanho do vetor (5) e índice solicitado (10). Encerramento controlado, sem corrupção de memória.

b) Comparação com Exercício 1 (C)

Em C, o acesso além dos limites é silencioso e resulta em corrupção de memória ou comportamento indefinido difícil de rastrear. Em Rust, o panic imediato com mensagem informativa torna o bug imediatamente visível e rastreável. Para casos críticos de performance, *get()* retorna *Option* e *get_unchecked()* em bloco *unsafe* desabilita a verificação explicitamente.

PARTE 3: ANÁLISE COMPARATIVA

Exercício 13 – Tabela Comparativa

Problema	C	Rust
Buffer overflow	<i>Possível — silencioso, comportamento indefinido</i>	<i>Impossível (safe) — panic com mensagem clara</i>
Dangling pointer	<i>Possível — apenas warning, crash em execução</i>	<i>Impossível — erro de compilação (lifetimes)</i>
Memory leak	<i>Possível — free esquecido, memória nunca devolvida</i>	<i>Improvável — Drop automático ao fim do escopo</i>
Double free	<i>Possível — corrupção do heap, crash</i>	<i>Impossível — ownership garante um único drop</i>
Null pointer deref	<i>Possível — segfault em execução</i>	<i>Gerenciado — Option força tratamento explícito</i>
Data race	<i>Possível — sem controle de aliasing com threads</i>	<i>Impossível (safe) — borrow checker proíbe aliasing mutável</i>

Exercício 14 – Reflexão

a) Por que C ainda é amplamente utilizado?

Apesar das vulnerabilidades conhecidas, C permanece dominante em sistemas de baixo nível por ao menos três razões fundamentais.

1. Legado e infraestrutura existente. Décadas de desenvolvimento produziram bilhões de linhas de código C sustentando sistemas operacionais (Linux, Windows, macOS), drivers de hardware, bibliotecas fundamentais (libc, OpenSSL, SQLite) e firmware embarcado. Reescrever esse código seria economicamente inviável e introduziria riscos imensos. O ecossistema de ferramentas, compiladores (GCC, Clang) e depuradores maduros para C é incomparável em maturidade.

2. Portabilidade e suporte universal a hardware. C funciona em praticamente qualquer arquitetura — de microcontroladores de 8 bits a supercomputadores. Em sistemas embarcados com poucos KB de RAM e sem sistema operacional (bare metal), C é frequentemente a única opção viável, pois gera código extremamente compacto sem dependências de runtime. Rust ainda não tem suporte para muitas arquiteturas legadas.

3. Simplicidade conceitual e controle total. C é uma linguagem pequena com modelo mental direto: "o código faz exatamente o que você escreve". Rust tem curva de aprendizado significativamente mais íngreme devido ao borrow checker. Certificações de segurança funcional (IEC 61508, DO-178C, ISO 26262) têm metodologias estabelecidas para C, mas ainda emergentes para Rust.

b) O sistema de ownership torna Rust mais difícil? Quais trade-offs?

Sim, o sistema de ownership é o maior obstáculo no aprendizado de Rust. Programadores vindos de linguagens com garbage collector ou de C/C++ precisam mudar fundamentalmente a forma de raciocinar sobre memória e propriedade de dados.

A dificuldade se manifesta no borrow checker: é comum escrever código logicamente correto que o compilador rejeita com erros sobre lifetimes ou empréstimos conflitantes. A comunidade criou a expressão "fighting the borrow checker" para descrever esse fenômeno. A curva é especialmente íngreme nas primeiras semanas, e muitos desistem nesse período.

Os **trade-offs** são claros: uma vez que o código compila, tende a ser muito mais correto e seguro. O compilador elimina em tempo de compilação uma classe inteira de bugs que em C só seriam descobertos em produção, às vezes após incidentes graves. Rust também oferece abstrações de alto nível (traits, iteradores, pattern matching, tipos algébricos) que tornam o código expressivo e seguro simultaneamente. Para sistemas críticos, esse nível de garantia compensa amplamente a dificuldade inicial.

c) Quando escolher C vs. Rust — cenários concretos

Escolheria C quando: (1) Microcontroladores de 8/16 bits (AVR, PIC, MSP430) onde Rust não tem suporte ou o runtime mínimo é grande demais; (2) A equipe é experiente em C e o prazo é curto; reescrever não justifica o custo; (3) Extensões ou plugins de base de código C existente (drivers Linux, módulos Python/PHP); (4) Certificações funcionais (DO-178C para aviação, ISO 26262 para automotivo) com toolchains estabelecidos apenas para C.

Escolheria Rust quando: (1) Servidores de alta disponibilidade e sistemas onde segurança de memória é crítica (pagamentos, autenticação, criptografia); (2) Sistemas embarcados modernos (ARM Cortex-M, RISC-V) onde Rust já tem excelente suporte; (3) Programação concorrente e paralela de alta performance — as garantias eliminam data races por design; (4) Ferramentas de linha de comando (ripgrep, fd, bat) onde performance e corretude são igualmente importantes; (5) WebAssembly — Rust é a linguagem de referência para WASM; (6) Novos drivers de kernel Linux — Rust é oficialmente suportado desde 2022.